BFS — Parcours en largeur d'un arbre

Algorithme, trace et implementation en C++

Errami Youssef

21 octobre 2026

1. Principe

Le parcours en largeur (*Breadth-First Search*, BFS) explore un arbre *niveau par niveau*. Tous les noeuds situes a une distance d de la racine sont visites avant tout noeud situe a une distance $d+1$. Cette propriete est obtenue grace a une structure de file (FIFO) : les fils decouverts sont ajoutes *a l'arriere* de la file, et le prochain noeud traite est toujours celui qui a ete decouvert le plus tot, c'est-a-dire celui *a l'avant*.

Les quatre ingredients de l'algorithme sont :

— la **racine** : point de depart, seul noeud initialement dans la file ;
— la **file** (`std::queue`) : contient les noeuds decouverts mais non encore traites ;
— le **vecteur** `visited` : enregistre les noeuds deja extraits, dans l'ordre de visite ;
— les **fils** : ajoutes a l'arriere de la file seulement s'ils existent.

2. Arbre de test

On utilise l'arbre binaire suivant. La racine est **1** et l'arbre possede 8 noeuds repartis sur 4 niveaux.

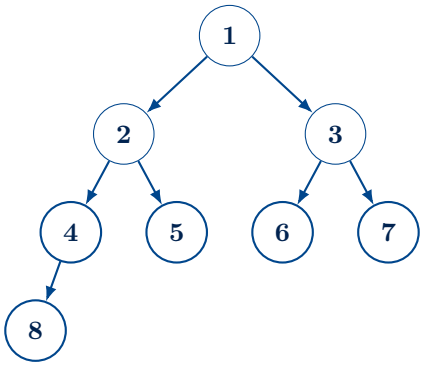


Figure 1. Arbre binaire utilise pour illustrer le parcours en largeur.

Les noeuds sont numerotes de facon a ce que l'ordre BFS soit l'ordre croissant : $1, 2, 3, 4, 5, 6, 7, 8$, regroupe par niveau : $\{1\}$, $\{2, 3\}$, $\{4, 5, 6, 7\}$, $\{8\}$.

3. Pseudocode

4. Trace d'execution

On applique l'algorithme a l'arbre precedent. A chaque etape on indique l'etat de la file (de la tete vers la queue) et celui du vecteur `visited` apres traitement du noeud.

Algorithme 1 BFS sur un arbre binaire

Entree : racine r de l'arbre
Sortie : liste `visited` des noeuds dans l'ordre de visite

```
1: file ← nouvelle file vide
2: visited ← nouveau vecteur vide
3: if r = nil then
4:     return visited
5: end if
6: file.push(r) // initialisation avec la racine
7: while file non vide do
8:     n ← file.front() // premier entre, premier sorti
9:     file.pop()
10:    visited.push_back(n.val)
11:    if n.left ≠ nil then
12:        file.push(n.left) // ajout a l'arriere
13:    end if
14:    if n.right ≠ nil then
15:        file.push(n.right)
16:    end if
17: end while
18: return visited
```

Etape	File (FIFO)	Visited	Action
0	[1]	[]	Push racine 1
1	[2, 3]	[1]	Pop 1 ; push 2 (G) ; push 3 (D)
2	[3, 4, 5]	[1, 2]	Pop 2 ; push 4 (G) ; push 5 (D)
3	[4, 5, 6, 7]	[1, 2, 3]	Pop 3 ; push 6 (G) ; push 7 (D)
4	[5, 6, 7, 8]	[1, 2, 3, 4]	Pop 4 ; push 8 (G) ; pas de droit
5	[6, 7, 8]	[1, 2, 3, 4, 5]	Pop 5 ; feuille
6	[7, 8]	[1, 2, 3, 4, 5, 6]	Pop 6 ; feuille
7	[8]	[1, . . . , 7]	Pop 7 ; feuille
8	[]	[1, . . . , 8]	Pop 8 ; feuille

A l'issue de l'etape 8 la file est vide : l'algorithme s'arrete. Le vecteur `visited` contient $[1, 2, 3, 4, 5, 6, 7, 8]$, soit exactement un noeud de chaque niveau avant de passer au suivant.

4.1 Observation cle

A l'etape 2, le noeud 3 reste en tete de file alors que les noeuds 4 et 5 viennent d'etre ajoutes. Le noeud 3 sera donc traite *avant* ses propres neveux : c'est ce comportement FIFO qui garantit le parcours niveau par niveau. Une pile (LIFO) produirait un parcours en profondeur (DFS) a la place.

5. Implementation C++

```
#include <iostream>
#include <vector>
#include <queue>

struct Node {
    int val;
    Node* left;
```

```
    Node* right;
    Node(int v) : val(v), left(nullptr), right(nullptr) {}
};

std::vector<int> BFS(Node* racine) {
    std::vector<int> visited;
    if (!racine) return visited;

    std::queue<Node*> file;
    file.push(racine);

    while (!file.empty()) {
        Node* n = file.front();
        file.pop();
        visited.push_back(n->val);

        if (n->left)  file.push(n->left);
        if (n->right) file.push(n->right);
    }
    return visited;
}
```

Les operations `file.push` et `file.front()` + `file.pop()` sont en $\mathcal{O}(1)$ amorti, ce qui donne une complexite totale lineaire.

6. Proprietes et complexite

Propriete	Valeur	Remarque
Complexite temps	$\mathcal{O}(V)$	Chaque noeud traite une fois
Complexite espace	$\mathcal{O}(V)$	File + vecteur visited
Memoire max de file	$\mathcal{O}(W)$	W = largeur max de l'arbre
Ordre de visite	par niveau croissant	Distance a la racine croissante
Premier chemin court	oui (sur graphe)	Propriete fondamentale du BFS
Recursif	non	Implementation iterative

Le point cle est que le BFS visite les noeuds par distance croissante a la racine. Cette propriete lui permet, sur un graphe general, de trouver le plus court chemin en nombre d'aretes, ce que le DFS ne garantit pas.

7. Conclusion

Le BFS est un algorithme simple mais dont la correction repose entierement sur l'usage d'une file FIFO. Trois idees suffisent a le resumer : *initialiser* la file avec la racine, *defiler* le noeud de tete, puis *enfiler* ses fils a l'arriere. Toute la puissance de l'algorithme vient de ce point : un noeud decouvert plus tot est traite plus tot, ce qui impose un parcours en largeur et non en profondeur. La trace de la section 4 illustre ce mecanisme de facon concrete : a chaque etape la file contient exactement les noeuds en cours d'exploration et le vecteur `visited` enregistre l'ordre definitif.